

Test Automation Generation (TAG) App | LLM (RAG) Project

GOAL :

To test a system that accurately generates test scripts and explores methods effectively. The system should ensure that, once a test case is generated, it creates proper import statements, correctly structures classes and objects, and makes appropriate method calls.

APPROACH :

Our approach begins with cleaning data using Abstract Syntax Tree (AST) techniques, converting it into a structured JSON format for consistency and organization. This cleaned data is indexed using the FAISS library to create a robust vector database that facilitates efficient similarity searches. Upon inputting a test case, the system performs a similarity search within this vector database to identify the closest matching data, which is then retrieved and stored in a list. The original test case is appended to this list of relevant data, and this combined input is fed into the language model (LLM). This process ensures the LLM generates test scripts that are accurate and contextually relevant. By integrating advanced data cleaning methods and efficient similarity search algorithms, this approach aims to enhance the reliability of automated test script generation, ultimately improving the accuracy and relevance of the generated scripts.

Data Cleaning: Abstract Syntax Tree (AST) Techniques

- **Purpose:** To convert unstructured data into a structured format.
- **Process:** The source code is parsed into an AST, which represents the code as a tree structure. Each node in the tree denotes a construct occurring in the source code.
- **Conversion to JSON:** The AST is then converted into a JSON format. This step ensures consistency and organization, making it easier to process and retrieve relevant information. The JSON format is particularly useful for further

processing steps as it allows for easier manipulation and querying of the data structure.

FAISS Library: Similarity Search

Purpose: To index and search large-scale datasets efficiently.

- **Indexing:** The cleaned data (in JSON format) is indexed using the FAISS (Facebook AI Similarity Search) library. FAISS is optimized for fast and accurate similarity searches on high-dimensional vectors.
- **Vector Creation:** Each test case and historical data entry is converted into a vector representation. This involves encoding the data using techniques such as word embeddings or sentence embeddings.
- **Similarity Search:** Upon inputting a new test case, the system performs a similarity search within the FAISS-indexed vector database to identify the closest matching historical data. This search leverages the high efficiency and accuracy of FAISS in handling large datasets.

Language Model (LLM) Integration:

- **Combined Input:** The original test case and the retrieved relevant data from the similarity search are combined. This combined input includes the new test steps and the contextually relevant historical data.
- **Feeding to LLM:** This combined input is then fed into the language model (LLM). The LLM processes this input to generate test scripts that are both accurate and contextually relevant.
- **Output Generation:** The LLM generates test scripts that include proper import statements, correctly structured classes and objects, and appropriate method calls. The accuracy and relevance of these scripts are enhanced by the context provided through the similarity search phase.

Techniques to Mitigate Overfitting and Hallucination:

- **Overfitting:** To prevent the model from focusing too much on the retrieved methods and not directly addressing the query, we employ regularization techniques and ensure a diverse and representative dataset.

- **Hallucination:** We reduce hallucination by using clean and structured data, ensuring that the model generates relevant and accurate information. Cross-validation and iterative refinement of the model are also used to improve the reliability of the generated scripts.

Why we are not using the Lang Chain's RetrievalQA ?

LangChain's Retrieval QA Approach

In LangChain's Retrieval QA approach, the process is as follows:

1. **Enter Test Case:** The test case with steps is provided.
2. **LLM Processing:** The LLM processes the test case directly.
3. **FAISS Vector Database:** The FAISS vector database is used to retrieve the closest methods based on similarity search.
4. **Retrieval Data:** The retrieved data (methods) is combined with the test case.
5. **Generate Test Script:** The combined input is used to generate the test script.

In this approach, the LLM processes the test case and retrieved documents separately, and the integration happens at a later stage. While the prompt is detailed, guiding the LLM through the test case description, template, and a sample script, this approach may not fully integrate the test steps with the retrieved data, potentially leading to less relevant or hallucinated responses.

Direct Approach

In the direct approach, the process is as follows:

1. **Enter Test Case:** The test case with steps is provided.
2. **FAISS Vector Database:** The FAISS vector database is used to search for the closest methods based on similarity search.
3. **Methods and Test Steps:** The methods retrieved by FAISS and the test steps are combined and stored in a list.
4. **LLM Processing:** This combined list (test steps and methods) is passed to the LLM.

5. **Generate Test Script:** The LLM uses this comprehensive input to generate the test script.

This approach ensures that the LLM has a complete understanding of the context by combining both the test steps and the relevant retrieved data before passing them into the LLM. By providing a structured and integrated prompt, the LLM can focus on relevant information from both sources, reducing the likelihood of hallucinated responses. This method also offers more control over the input, leading to more precise and reliable outputs.

Key Differences and Benefits

1. **Integration of Data:** In the direct approach, test steps and retrieved methods are combined and passed together into the LLM, ensuring a holistic understanding of the context.
2. **Focus and Relevance:** The LLM can focus on relevant information from both the test steps and retrieved data, reducing irrelevant or hallucinated responses.
3. **Control:** The direct approach provides more control over the input, leading to more accurate and relevant test scripts.
4. **Reduced Hallucination:** By integrating the data before LLM processing, the direct approach minimizes the chances of the LLM generating unrelated or incorrect information.

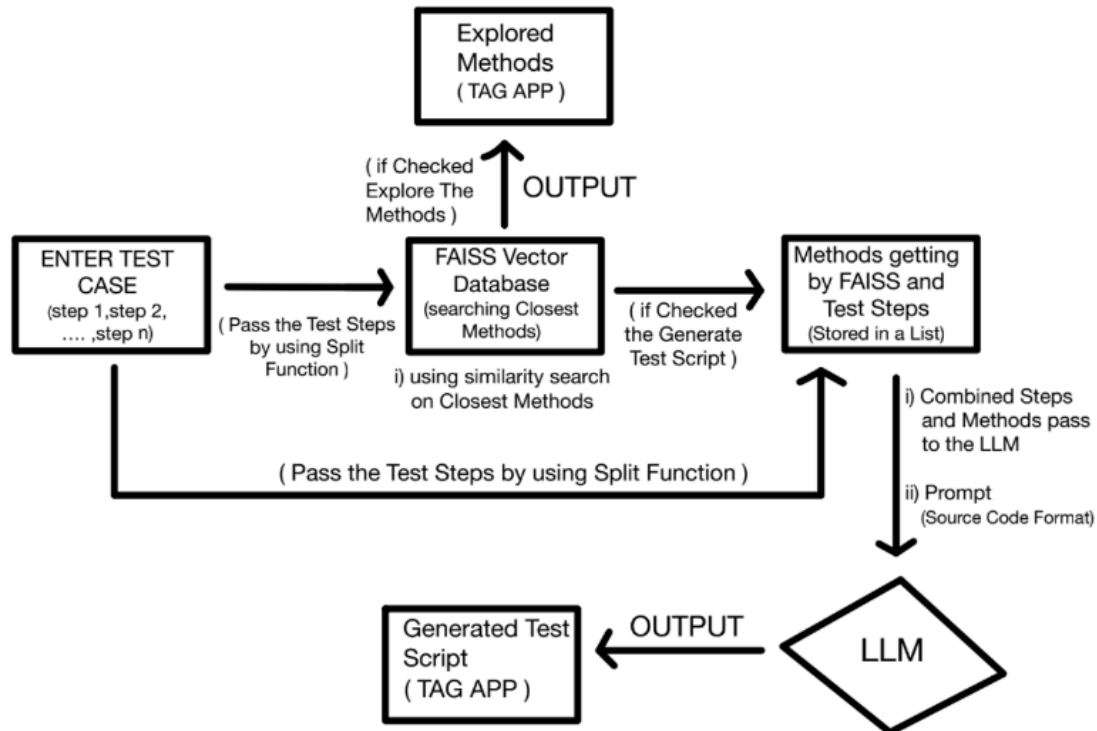
How the LLM works using Prompt ?

→ **Prompt written in the code part for generating testscript :**

- **Generating Both Hypothetical and Closest Methods:** Initially, the LLM generates both hypothetical methods based on the test steps and retrieves the closest existing methods from the FAISS vector database.
- **Verification of Methods:** Once both the hypothetical methods and the closest methods are generated, the LLM verifies which methods are most appropriate for each test step. This verification process ensures that the methods used in the final test script are the closest match to the required functionality.

- **Preference for Verified Methods:** The LLM gives more preference to the verified methods that best match the test steps. This means that the methods selected for the final test script are those that have been determined to be the most relevant and accurate for each test step.

Flow Chart :



Compare between Before and After :

i) Test Script

→ When we wrote the same Test Steps for given results for generating the Test script.

Test Steps :

- step 1 : start odv client
- step 2 : login to basidi
- step 3 : open 110kv vienna
- step 4 : close 110kv vienna
- step 5: close basidi

```
# Required module imports
from modules.basidi.basidi_page import BasidiPage
from modules.basidi.window33kv.overview33kvvienna_page import overview33kvvienna_page
from modules.basidi.tab110kv.tab110kv_page import Tab110kvPage

def init():
    # Initialize BasidiPage, overview33kvviennapage, and Tab110kvPage
    basidi_page = BasidiPage()
    overview_page = overview33kvviennapage()
    tab_page = Tab110kvPage()

def main():
    # Step 1: Start odvclient
    # Assuming there's a function to start odvclient
    start_odvclient()

    # Step 2: Login to basidi
    # Assuming there's a function to login to basidi
    login_basidi()

    # Step 3: Open 110kv-vienna
    basidi_page.open_110kv_vienna()

    # Step 4: Close 110kv Vienna
    # Assuming there's a function to close 110kv vienna
    close_110kv_vienna()

    # Step 5: Close basidi
    # Assuming there's a function to close basidi
    close_basidi()

def cleanup():
    # Assuming there's a function to cleanup after test
    cleanup_after_test()
```

BEFORE

We are getting correct generated test Scripts at this stage .

```
# -*- coding: utf-8 -*-
"""
Jira ticket: <>

Desc: step 1 : start odv client
      step 2 : login to basidi
      step 3 : open 110kv vienna
      step 4 : close 110kv vienna
      step 5: close basidi
"""

# import necessary modules
source(findFile("scripts", "odvclient/odvclient_page.py"))
source(findFile("scripts", "basidi/basidi_page.py"))
source(findFile("scripts", "basidi/listofdisplays/viennasubstation_page.py"))

# create objects
odv_client = OdvClientPage()
basidi = BasidiPage()
vienna_substation = ViennaSubstationPage()

def init():
    """ init function """
    test.log("init method execution started")
    # Step 1: Start odv client
    odv_client.start_odvclient()
    test.log("init method execution completed")

def main():
    """ main function """
    test.log("Main method execution started")
    # Step 2: Login to basidi
    #basidi.login_basidi() # Hypothetical function, not exist in modules
    # Step 3: Open 110kv vienna
    basidi.open_110kv_vienna_substation()
    # Step 4: Close 110kv vienna
    vienna_substation.clickOn_220kv_worldmap()
    test.log("Main method execution completed")

def cleanup():
    """ clean up function """
    test.log("clean up method started")
    # Step 5: Close basidi
    basidi.close_basidi()
    test.log("end of execution")
```

AFTER

ii) Explore Method

→ When we wrote the 'we need to create a job ' for given results

BEFORE :

✓ Explore the Methods

Please Enter your test case

we need to create a job in immhome

1. Method: navigate_IMMHomepage
Module: immhome\immhome_page.py
Description: This method is used to navigate to the IMM Homepage. This could be the first step in creating a job in immhome as the user needs to be on the IMM Homepage to start the job creation process.
2. Method: click_onthejob
Module: immhome\immhome_page.py
Description: This method is used to click on a specific job. It takes a job name as an argument. This could be used to select the job that needs to be created.
3. Method: filter_imm_job
Module: immhome\immhome_page.py
Description: This method is used to filter the created job. It takes a job name as an argument. This could be used to search for the newly created job and verify its creation.

AFTER:

✓ Explore the Methods

Please Enter your test case

we need to create a job



Explored Methods, Classes, and Modules:

```
[
  0 :
  [{"file_path": "immhome\\immhome_page.py", "class": "immhomepage", "function_name":
  "create_job", "implementation": "\n\ndef create_job(self, jobname, description):\n    '\n
  Will right click on Realtime (RT)\n        Will click on New job\n        will enter jobname,
  description and click on OK\n        Parameters:\n        jobname :
  string\n        description : string\n        '\n
  openContextMenu(waitForObject(self.datamodelobject.frame_tree_dataSetTree_mage_DataSetTree,
  'Realtime (RT)'), 92, 8, Qt.NoModifier)\n
  activateItem(waitForObject(self.datamodelobject.o_QMenu_realtime, 'New job'))\n
  type(waitForObject(self.datamodelobject.mage_NewJob_jobName_QLineEdit), jobname)\n
  type(waitForObject(self.datamodelobject.mage_NewJob_jobName_QLineEdit), '<Tab>')\n
  type(waitForObject(self.datamodelobject.mage_NewJob_jobDescEdit_QTextEdit), description)\n
  clickButton(waitForObject(self.datamodelobject.mage_NewJob_OK_QPushButton, 25000))\n
  waitForObject(self.datamodelobject.station_URL_TxtBox, 90000)\n    snooze(5)\n
  self.setToFixedView()\n"}]
```

- I currently search the methods using the FAISS vector database. This approach has proven to be more accurate than the previous method of using a llm , which occasionally missed some methods. The new FAISS-based search significantly improves the completeness and correctness of the method retrieval.